

ENV 6128

Smart Air Quality Monitoring and Air Pollution Control

Fall 2024

BA1-216A & online

TuTh 3:00 – 4:15 PM

IoT Basics (Embedded System)

- A combination of computer hardware and software that is designed for specific function or functions within a larger system. Embedded system is fixed in capability and programmable features.



Microcontroller and Microprocessor

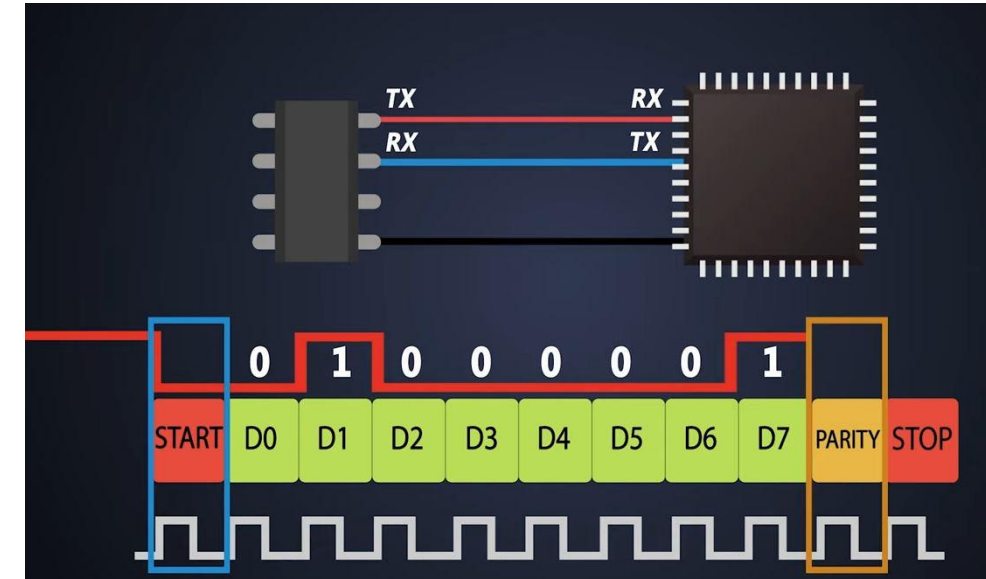
Microcontroller	Microprocessor
Less memory space provided	More memory space provided
16MHz-500MHz	1GHz+
Fewer features	More features
Smaller, weaker	Stronger, heavy duty
Less expensive	Relatively Expensive
Write a program on a host machine (your laptop), place in the microcontroller memory	Directly write on it
C/C++	Python
Arduino	Raspberry Pi

Universal asynchronous receiver-transmitter (UART)

Two number of wires used in UART communication protocol.

1. Transmitter (TX)
2. Receiver (RX)

- Transmitter Line (TX): Send data to the Receiver Line at a specified baud rate (bit/sec speed)
- Receiver Line (RX): The RX wire reads the incoming stream of bits and interprets the data that arrives serially at the specified baud rate.

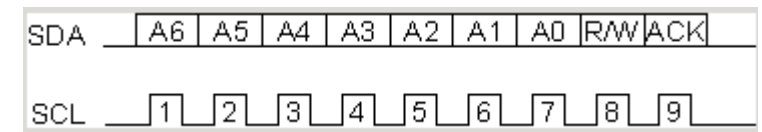
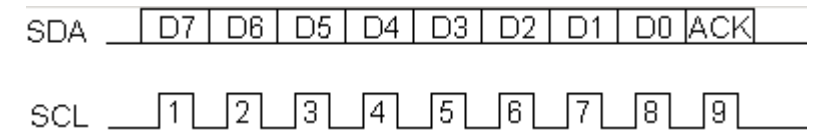
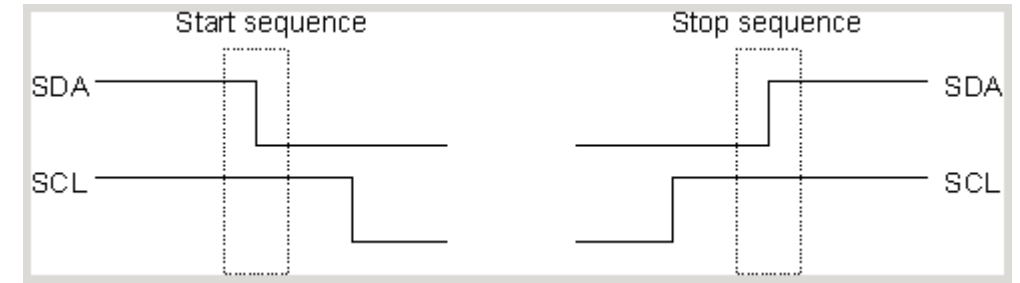


Inter-Integrated Circuit (I2C)

Two number of wires used in I2C communication protocol.

1. Serial Clock Line
2. Serial Data Line

- Serial Clock (SCL): Synchronization (Timing) of data sending
- Serial Data (SDA): Actual data send/receive



Arduino UNO data sheet

Features

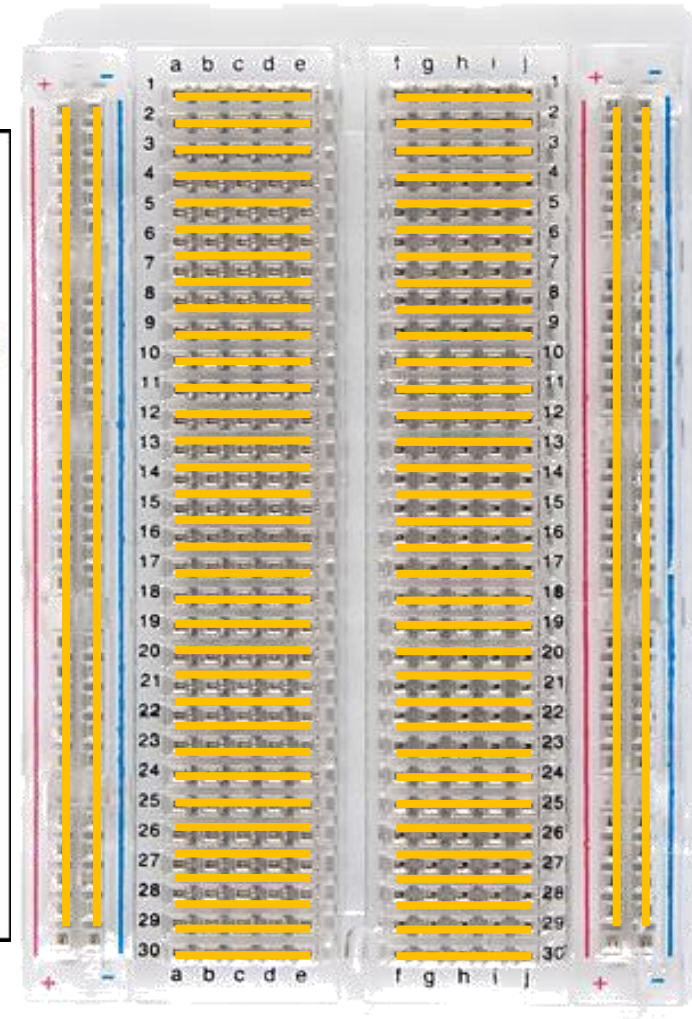
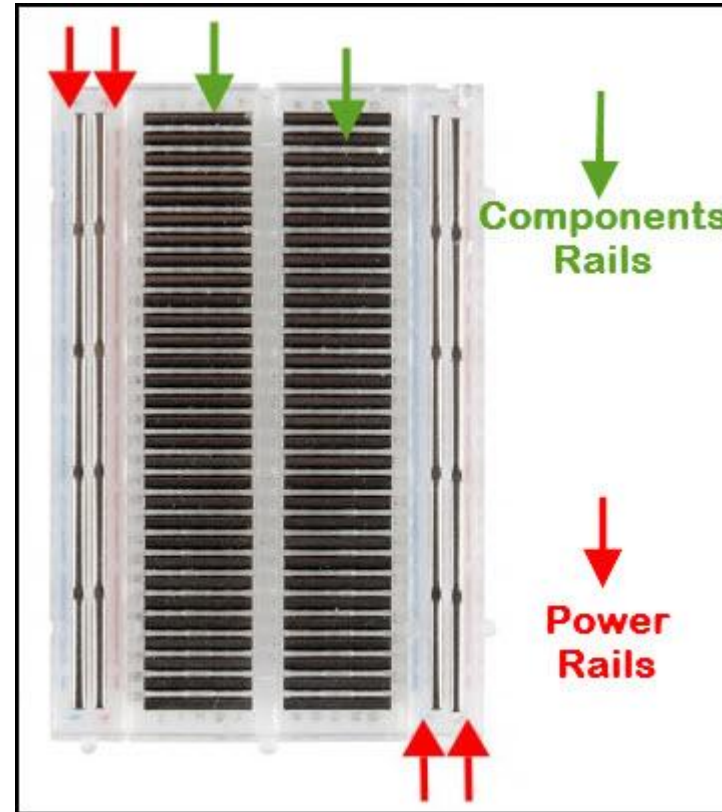
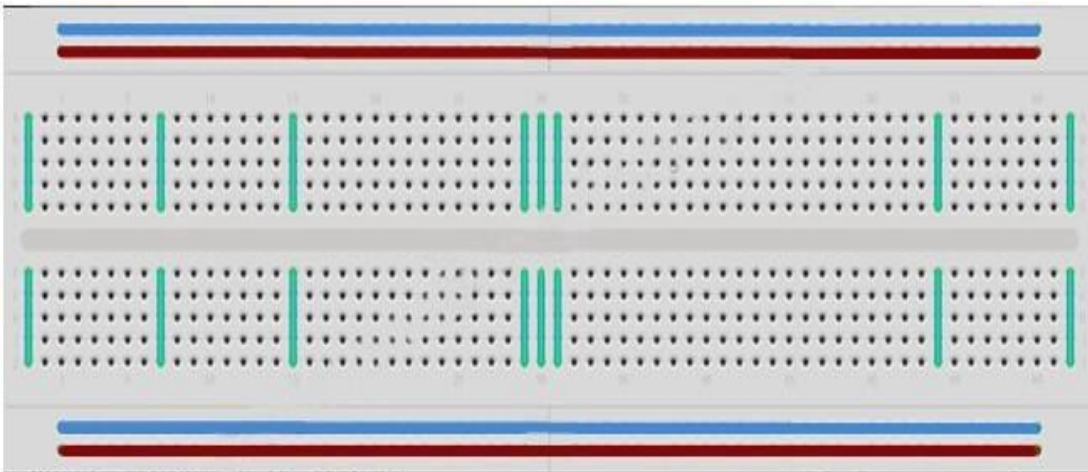
- **ATMega328P Processor**
 - **Memory**
 - AVR CPU at up to 16 MHz
 - 32KB Flash
 - 2KB SRAM
 - 1KB EEPROM
 - **Security**
 - Power On Reset (POR)
 - Brown Out Detection (BOD)
 - **Peripherals**
 - 2x 8-bit Timer/Counter with a dedicated period register and compare channels
 - 1x 16-bit Timer/Counter with a dedicated period register, input capture and compare channels
 - 1x USART with fractional baud rate generator and start-of-frame detection
 - 1x controller/peripheral Serial Peripheral Interface (SPI)
 - 1x Dual mode controller/peripheral I2C
 - 1x Analog Comparator (AC) with a scalable reference input
 - Watchdog Timer with separate on-chip oscillator
 - Six PWM channels
 - Interrupt and wake-up on pin change
- **ATMega16U2 Processor**
 - 8-bit AVR® RISC-based microcontroller
- **Memory**
 - 16 KB ISP Flash
 - 512B EEPROM
 - 512B SRAM
 - debugWIRE interface for on-chip debugging and programming
- **Power**
 - 2.7-5.5 volts



Useful Links

1. Download Arduino IDE: [Software | Arduino](#)
2. Documentation: [UNO R3 | Arduino Documentation | Arduino Documentation](#)
3. Tutorials: [Built-in Examples | Arduino Documentation | Arduino Documentation](#)

Breadboard



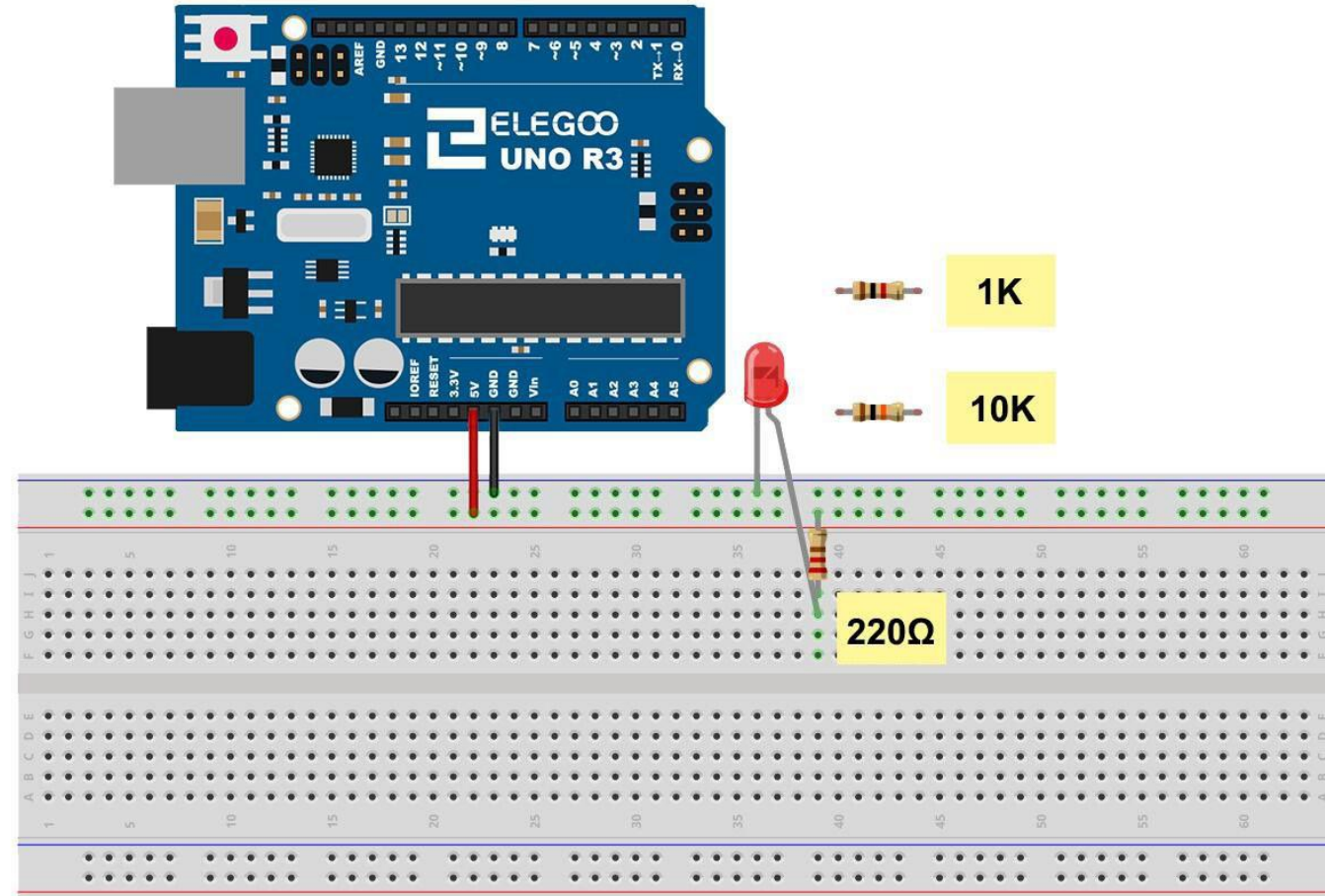
Used to prototype circuit
Connect multiple sensors to Arduino at the same time

Use of Breadboard (Blink and Connect LED)

Lessons 1, 2, and 3 from ELEEGO Super Starter Manual

We will upload the example blink code and see the Red LED using

1. Resistor (220 ohm, 1k ohm and 10k ohm)
2. Jumper Wire (M-M)
3. Red LED



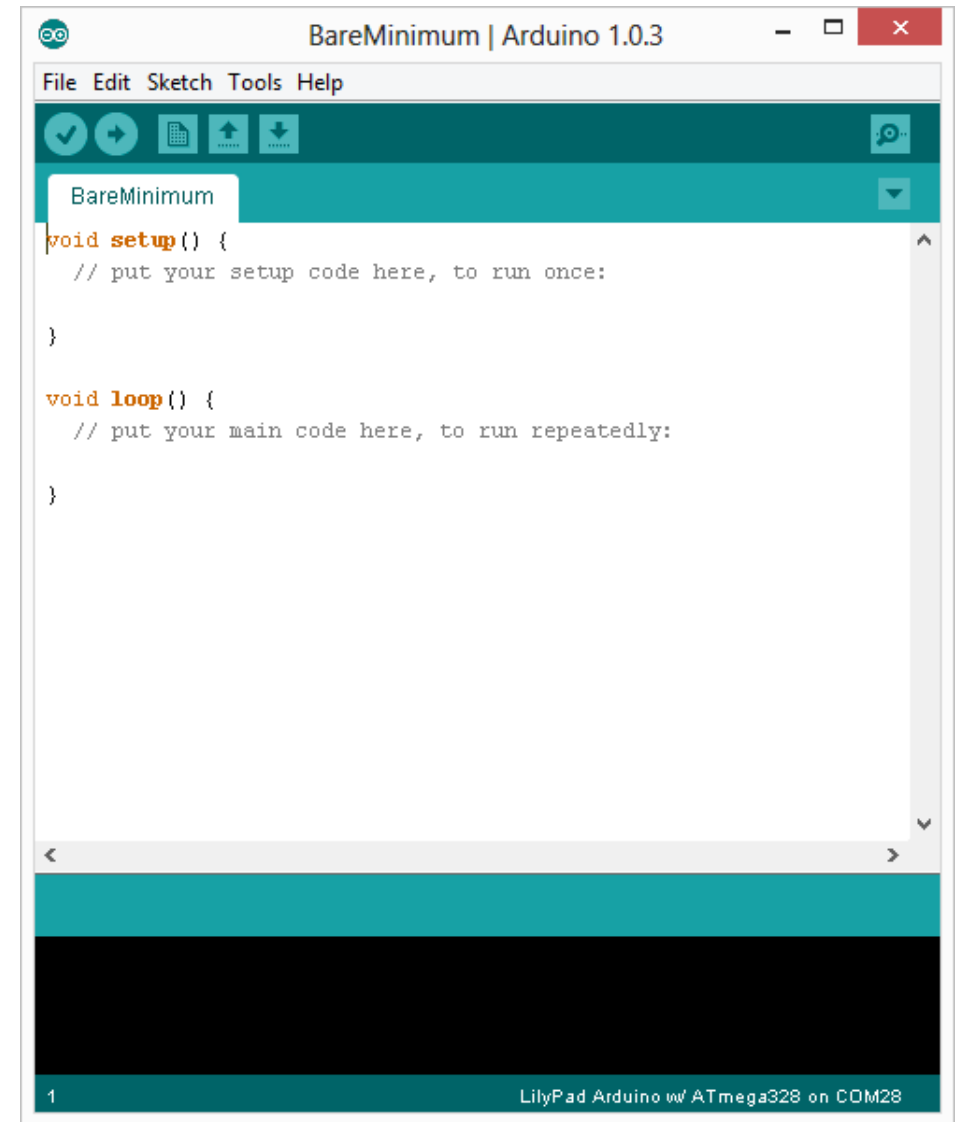
Arduino Programming

- C and C++ Programming Language Utilized

Void Setup – Initialize Code Setup at the beginning of the program (just once)

Void Loop – Program inside it will run at a loop for indefinite amount of time

// or /* */ used for comment



Arduino Programming

- pinMode – Configure a pin as input/output
- digitalWrite – Write a high/low value of voltage
- digitalRead – Read a high/low value of voltage
- Delay – Pause for a period before next line execution
- Include – Include outside library on your sketch
- Serial – Communicate between Arduino and other Sensors
- Serial.print – Prints data to the serial port as human-readable text.
- Serial.println - Serial.print + New Line

```
30 // the loop function runs over and over again forever
31 void loop() {
32   digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the volt
33   delay(500); // wait for a second
34   digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the vo
35   delay(500); // wait for a second
36 }
```

Fundamental Data Types

- `char` – Store a character value (e.g., `char myChar = 'A';`)
- `int` – Store numbers (e.g., `int countUp = 1;`)
- `float` – Store numbers that contain decimal points (e.g., `float a = 1.117`)
- `double` – Double precision floating point number, can store with higher precision (e.g., `double pi = 3.14159`)
- `Uint16_t` – 16-bit integer unsigned (non-negative)

Relational and Logical Operators

Operator	Description	Example
>	Greater than	7 > 5 is TRUE
<	Less than	7 < 5 is FALSE
>=	Greater than or equal	2 >= 2 is TRUE
<=	Less than or equal	2 <= 1 is FALSE
==	Equal to	6 == 6 is TRUE
!=	Not equal to	6 != 6 is FALSE
!	LOGICAL NOT operator	<pre>if (! aqi.read(&data)) { Serial.println("Could not read from AQI") }</pre>

If/else Statement

if (Condition) {
do if the condition is true
}

```
int Student1_Marks = 92;  
int Student2_Marks = 20;  
  
void setup() {  
  Serial.begin(9600);  
  
  if (Student1_Marks > 33){  
    Serial.println("The student1 marks is greater than 33. He passed the exam.");  
  }  
}
```

If/else Statement

```
if (condition1) {  
    // do if the condition is True  
}  
else if (condition2) {  
    // do if the condition is True  
}  
else {  
    // do if none of the above condition is True  
}
```

```
int Marks = 95;  
char Grade;  
  
void setup() {  
    Serial.begin(9600);  
  
    if (Marks <= 33){  
        Grade = 'E';  
    }  
    else if (Marks <= 40){  
        Grade = 'D';  
    }  
    else if (Marks <= 60){  
        Grade = 'C';  
    }  
    else if (Marks <= 80){  
        Grade = 'B';  
    }  
    else if (Marks <= 100){  
        Grade = 'A';  
    }  
}
```

If/else Statement

```
if (someCondition) {  
  // do if the condition is true  
} else {  
  // do stuff if the condition is false  
}
```

```
int Student1_Marks = 92;  
int Student2_Marks = 20;  
  
void setup() {  
  Serial.begin(9600);  
  
  if (Student1_Marks > 33){  
    Serial.println("The student1 marks is greater than 33. He passed the exam.");  
  }  
  else{  
    Serial.println("The student1 marks is less than or equal to 33. He failed the exam.");  
  }  
}
```

For/While Loop

For loop: Repeat a block of statements enclosed in curly braces.

While loop: loop continuously, and infinitely, until the expression inside the parenthesis, () becomes false.

```
1  for (int x = 2; x < 100; x = x * 1.5) {  
2      println(x);  
3  }
```

```
1  var = 0;  
2      while (var < 200) {  
3      // do something repetitive 200 times  
4      var++;  
5  }
```

Project 1: Plantower Particle Sensor

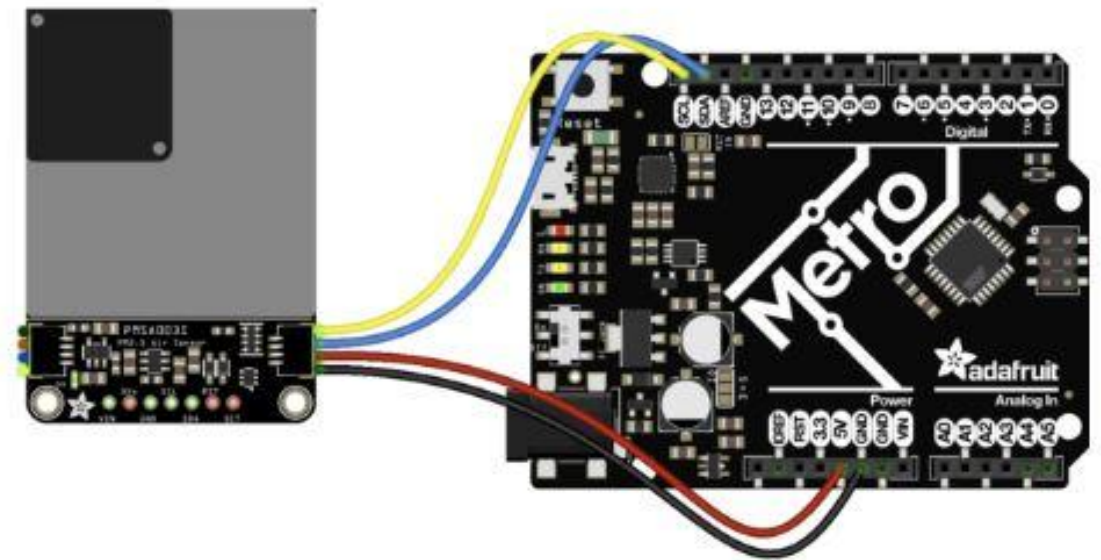
- Optical Particle Counter
- Count Number of Particles. From number of Particles, gives Particle concentration



Project 1: Plantower Particle Sensor

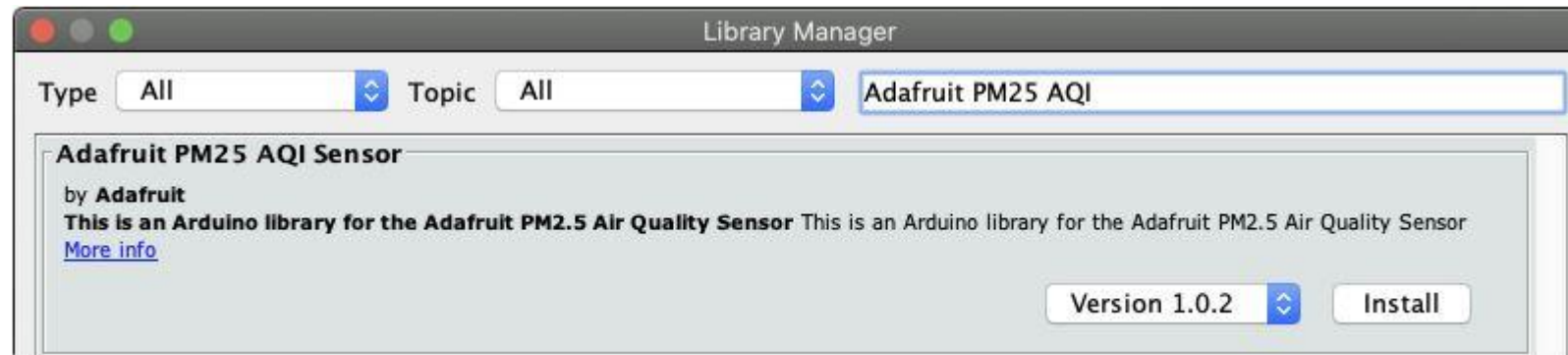
Wiring Diagram:

Sensor End	Microcontroller End
VCC (Red Wire)	5V
GND (Black Wire)	GND
SCL (Yellow Wire)	SCL
SDA (Blue Wire)	SDA



Project 1: Plantower Particle Sensor

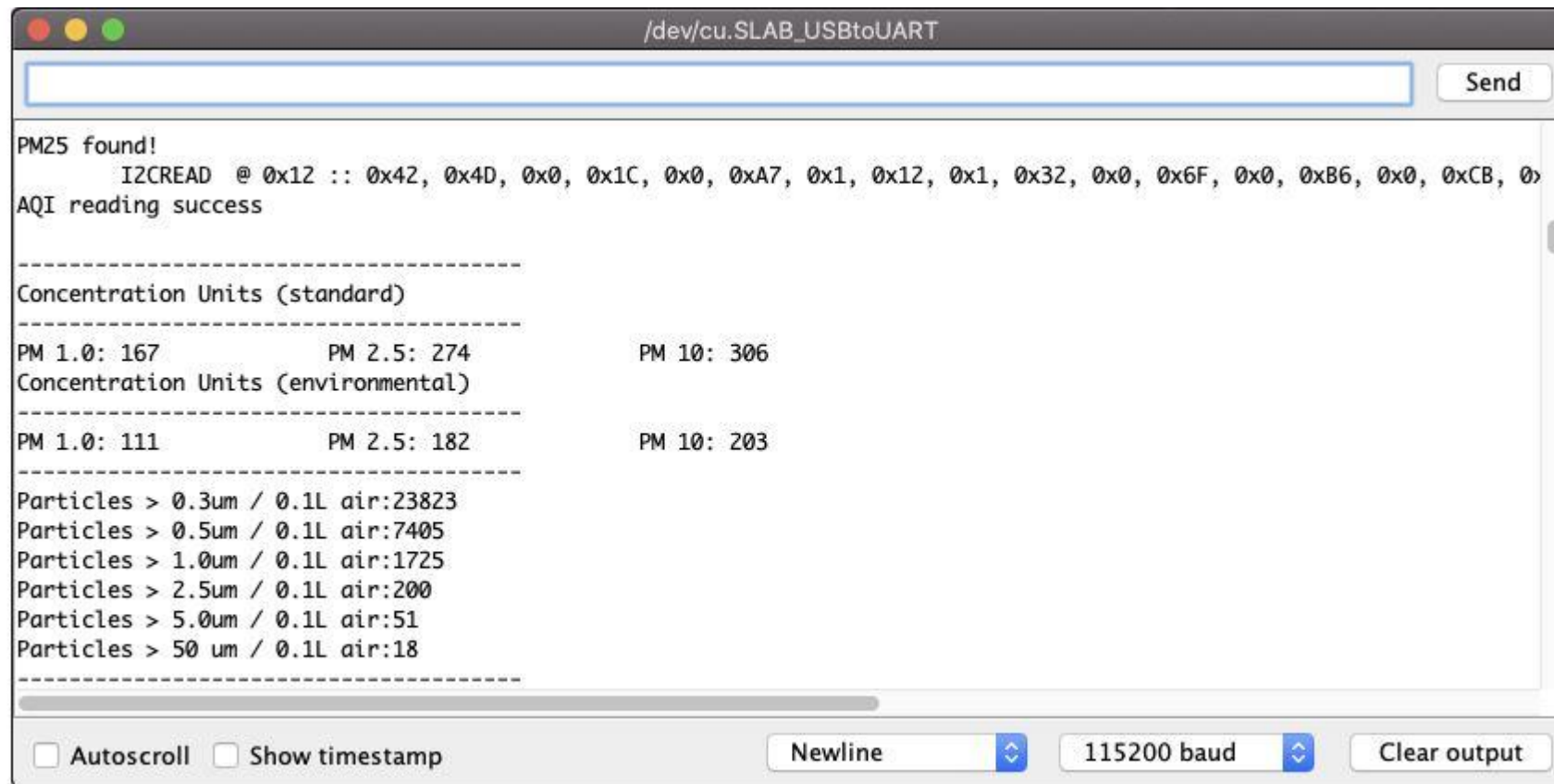
Library (Tools-Manage Libraries) Need to be installed: Adafruit PM25 AQI Sensor



Code and Library: [Arduino | Adafruit PMSA003I Air Quality Breakout | Adafruit Learning System](#)

Project 1: Plantower Particle Sensor

Output:



```
/dev/cu.SLAB_USBtoUART

PM25 found!
  I2CREAD @ 0x12 :: 0x42, 0x4D, 0x0, 0x1C, 0x0, 0xA7, 0x1, 0x12, 0x1, 0x32, 0x0, 0x6F, 0x0, 0xB6, 0x0, 0xCB, 0x
AQI reading success

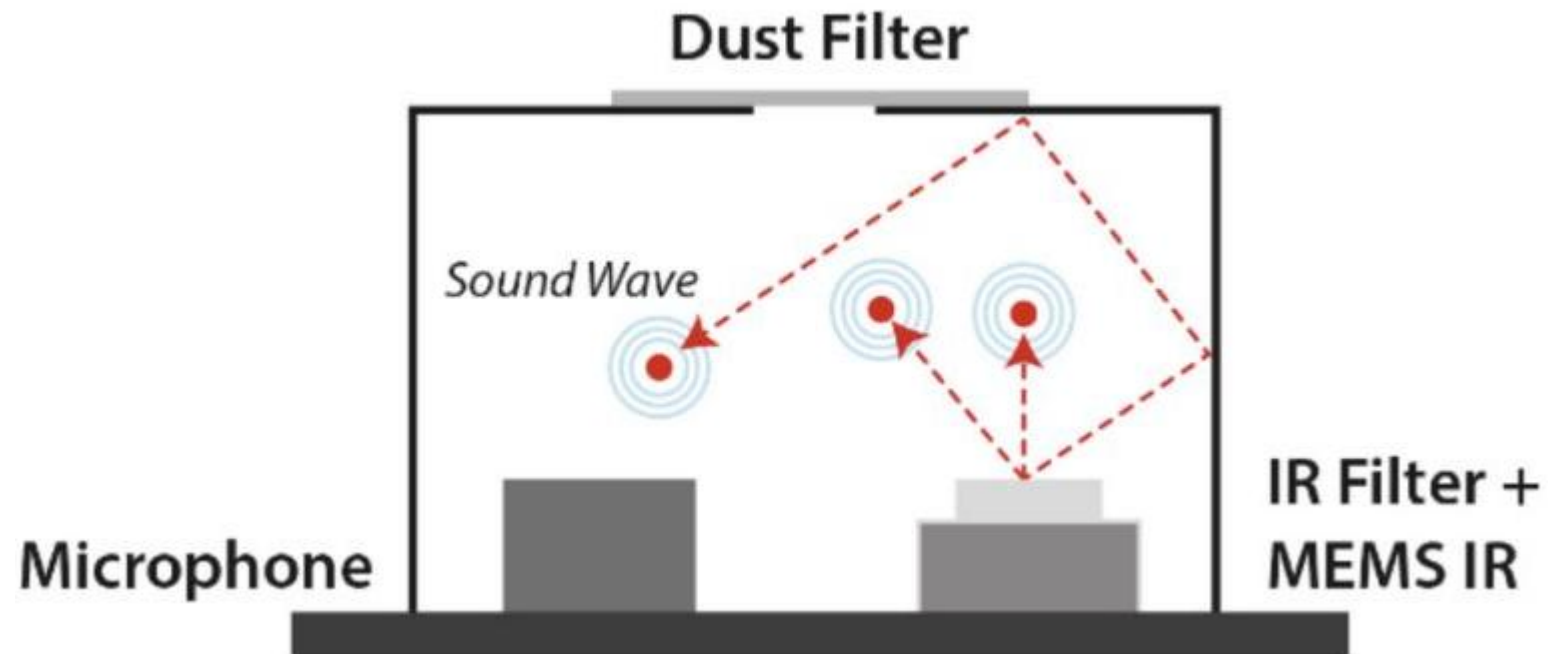
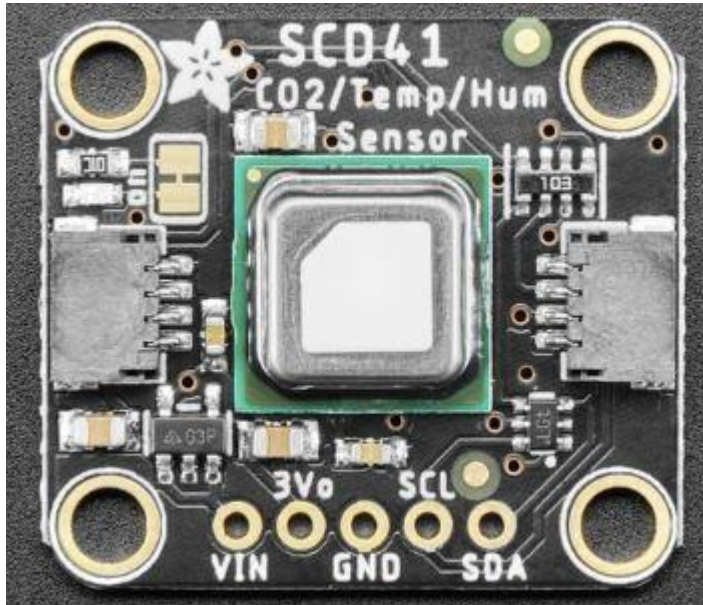
-----
Concentration Units (standard)
-----
PM 1.0: 167          PM 2.5: 274          PM 10: 306
Concentration Units (environmental)
-----
PM 1.0: 111          PM 2.5: 182          PM 10: 203
-----
Particles > 0.3um / 0.1L air:23823
Particles > 0.5um / 0.1L air:7405
Particles > 1.0um / 0.1L air:1725
Particles > 2.5um / 0.1L air:200
Particles > 5.0um / 0.1L air:51
Particles > 50 um / 0.1L air:18
-----

☐ Autoscroll ☐ Show timestamp
Newline 115200 baud Clear output
```

Code and Library: [Arduino](#) | [Adafruit PMSA003I Air Quality Breakout](#) | [Adafruit Learning System](#)

Project 2: SCD 41 CO2 Sensor

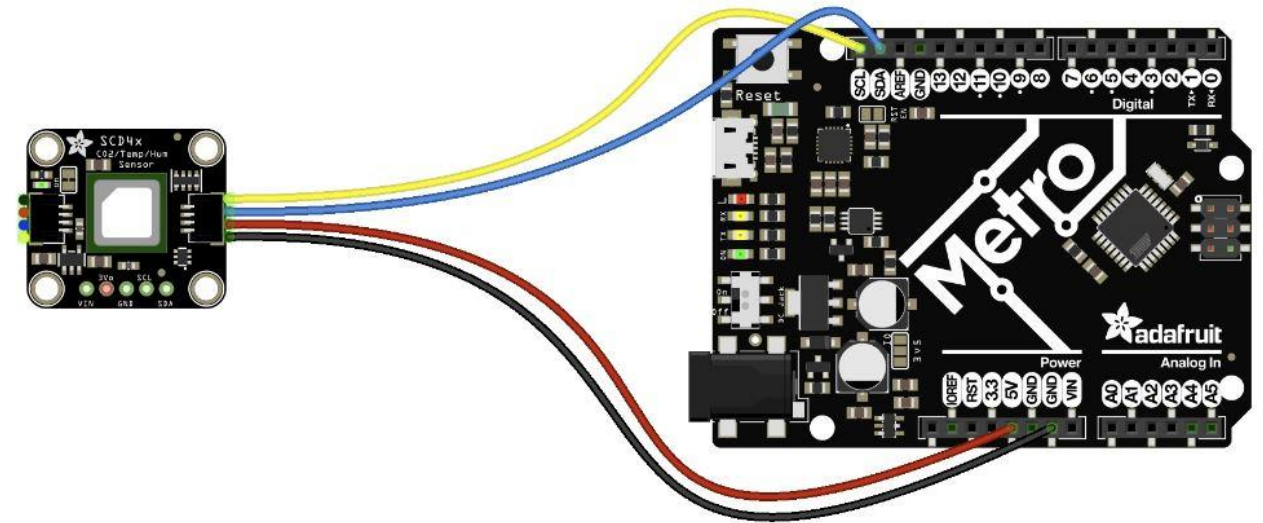
- Photoacoustic CO2 Sensor
- Detect the concentration of CO2 by the interaction of light and sound wave



Project 2: SCD 41 CO2 Sensor

Wiring Diagram

Sensor End	Microcontroller End
VCC (Red Wire)	5V
GND (Black Wire)	GND
SCL (Yellow Wire)	SCL
SDA (Blue Wire)	SDA



Project 2: SCD 41 CO2 Sensor

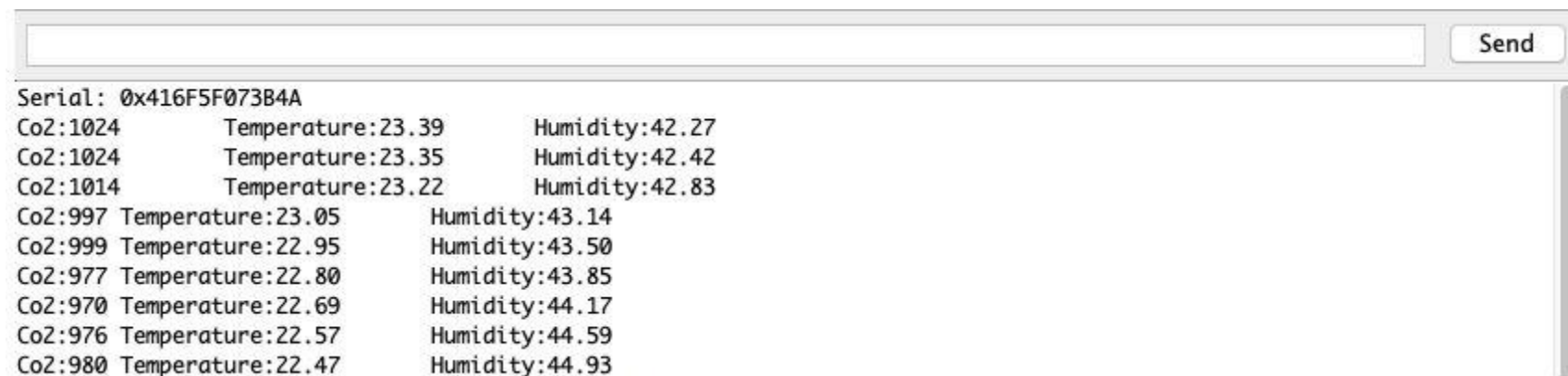
Library (Tools-Manage Libraries) Need to be installed: Sensirion I2C SCD 4x



Code and Library: [Arduino | Adafruit SCD-40 and SCD-41 | Adafruit Learning System](#)

Project 2: SCD 41 CO2 Sensor

Output:



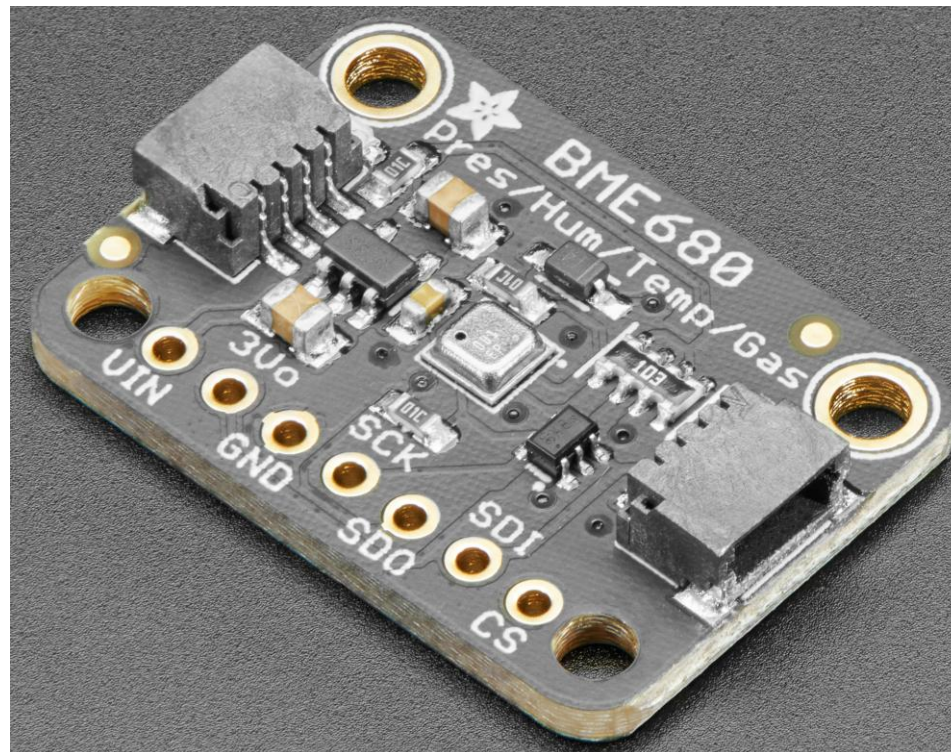
A screenshot of a serial monitor window. At the top, there is a text input field and a 'Send' button. Below the input field, the serial data is displayed. The first line shows the device's serial number: 'Serial: 0x416F5F073B4A'. Subsequent lines show sensor readings in a tabular format with three columns: CO2 concentration, Temperature, and Humidity. The data is as follows:

Co2	Temperature	Humidity
1024	23.39	42.27
1024	23.35	42.42
1014	23.22	42.83
997	23.05	43.14
999	22.95	43.50
977	22.80	43.85
970	22.69	44.17
976	22.57	44.59
980	22.47	44.93

Code and Library: [Arduino](#) | [Adafruit SCD-40 and SCD-41](#) | [Adafruit Learning System](#)

Project 3: Bosch BME 680 VOC Sensor

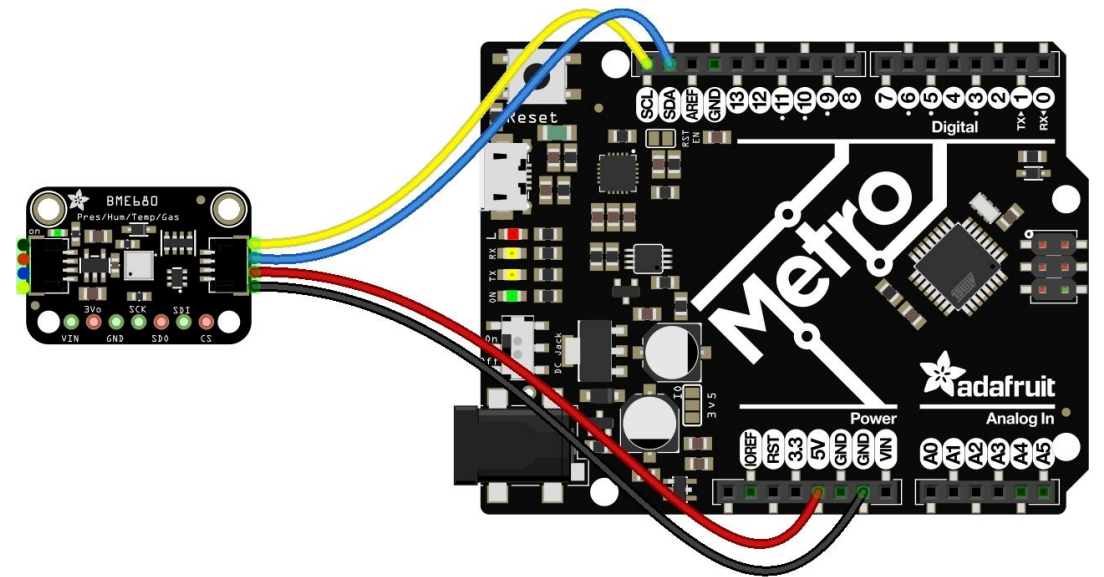
- Metal Oxide Semiconductor Technology
- Conductivity changes when the metal oxide surface comes in contact with the target gas (VOC)



Project 3: Bosch BME 680 VOC Sensor

Wiring Diagram

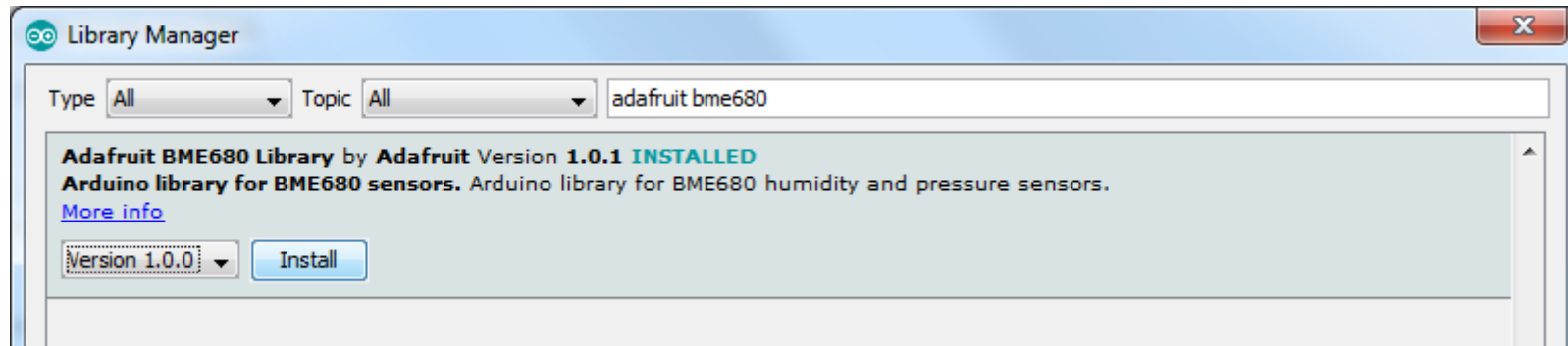
Sensor End	Microcontroller End
VCC (Red Wire)	5V
GND (Black Wire)	GND
SCL (Yellow Wire)	SCL
SDA (Blue Wire)	SDA



fritzing

Project 3: Bosch BME 680 VOC Sensor

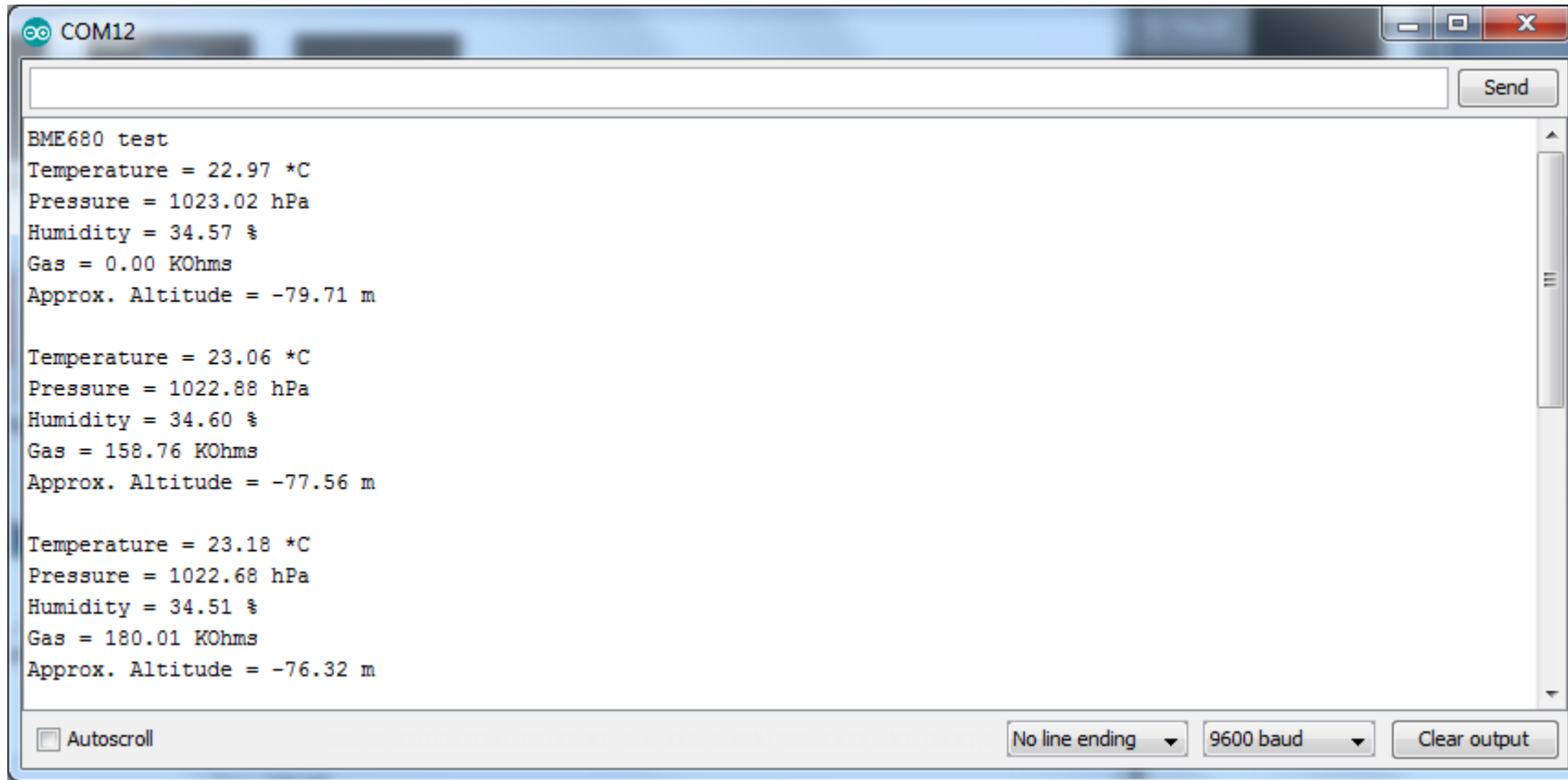
Library (Tools-Manage Libraries) Need to be installed: Adafruit_BME680



Code and Library: [Arduino Wiring & Test](#) | [Adafruit BME680](#) | [Adafruit Learning System](#)

Project 3: Bosch BME 680 VOC Sensor

Output:



The screenshot shows a serial monitor window with the title 'COM12'. The output text is as follows:

```
BME680 test
Temperature = 22.97 *C
Pressure = 1023.02 hPa
Humidity = 34.57 %
Gas = 0.00 KOhms
Approx. Altitude = -79.71 m

Temperature = 23.06 *C
Pressure = 1022.88 hPa
Humidity = 34.60 %
Gas = 158.76 KOhms
Approx. Altitude = -77.56 m

Temperature = 23.18 *C
Pressure = 1022.68 hPa
Humidity = 34.51 %
Gas = 180.01 KOhms
Approx. Altitude = -76.32 m
```

At the bottom of the window, there are controls: an 'Autoscroll' checkbox, a 'No line ending' dropdown menu, a '9600 baud' dropdown menu, and a 'Clear output' button.

Code and Library: [Arduino Wiring & Test](#) | [Adafruit BME680](#) | [Adafruit Learning System](#)

Save Data to folder

- Download Putty ([Download PuTTY - a free SSH and telnet client for Windows](#))
- Select Serial Line, Speed and Destination name. Click Open

